

COMP1004-DBI

Databases and Interfaces

Module conveners:

Nazia Hameed, Kai Xu

Last lecture

- Structure query language(SQL)
- Creating tables
- Datatypes
- Constraints
- Drop table
- Data manipulation language
 - Insert/update/delete data
 - Select data

Today's lecture

- Select data
- Where clause
- Aggregate functions
- Group By and Having
- Joins
- Sub queries
- String functions

SELECT General Form

SELECT [DISTINCT | ALL]

{* | [columnExpression [AS newName]] [,...]}

FROM TableName [alias] [, ...]

[WHERE condition]

[GROUP BY columnList] **[HAVING**
condition]

[ORDER BY columnList]

Row selection (WHERE)

- You will rarely want all the rows of your tables
- To restrict rows that are to be retrieved, use WHERE clause
- Keyword WHERE is followed by a search condition
- There are five basic search conditions (or predicates using the ISO terminology) available
- Example of filtering conditions
 - Price < 40
 - name = 'John'
 - name <> 'John'
 - First = Last
 - (First = 'John') AND (Last = 'Smith')
 - (Mark < 40) OR (Mark > 70)
 - Mark BETWEEN 40 AND 70

```
SELECT [DISTINCT | ALL]
{* | [columnExpression [AS newName]] [...] }
FROM          TableName [alias] [, ...]
[WHERE        condition]
[GROUP BY    columnList] [HAVING condition]
[ORDER BY    columnList]
```

Search conditions

- Comparison
 - Compare the value of one expression to the value of another expression
- Range
 - Test whether the value of an expression falls within a specified range of values
- Set membership
 - Test whether the value of an expression equals one of a set of values
- Pattern match
 - Test whether a string matches a specified pattern.
- Null
 - Test whether a column has a null (unknown) value

Comparison search condition

- List all bouquets with a price greater than 20

```
SELECT *  
FROM bouquet  
WHERE price > 20;
```



bouquet_id	bouquet_name	price
1	A New Day	23.5
2	The flower of love	20.5
4	I love you	33.0
5	Pretty in pink	24.5
6	You are beautiful	23.0
8	Simply red	26.0
16	Summer sun	22.0

SQL comparison operators

- In SQLite, the following simple comparison operators are available:

Operator	Meaning
=	Equal to
<> or !=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

Compound comparison search condition

- List of customers who are either greater than 25 years or don't have phone number

SELECT *
FROM customer
WHERE age > 25 **or** phone is null;



Cust_id	name	phone	age	address
2	Ali Rehman	+4475567855344	45	65 oxford drive Nottingham NG...
3	Kristian Liam	+4475567845644	31	176 Groove drive London GH4 ...
4	Lizzi Young	+4474463545024	34	5 Victoria Street Leicester LE6...
5	Ashar Junaid	+4455784186264	65	14 Manor Road Bath BA78 5PA
7	Nyla Noor	NULL	25	38 Church Road Guildford GU4...
8	Ashar Junaid	+4455784186264	56	82 Melbourne Road Nottingha...

Range search condition (BETWEEN/NOT BETWEEN)

- BETWEEN test includes the endpoints of range
- List all bouquets with a price between 20 and 30

```
SELECT *  
FROM bouquet  
WHERE price BETWEEN 20 AND 30;
```



bouquet_id	bouquet_name	price
1	A New Day	23.5
2	The flower of love	20.5
5	Pretty in pink	24.5
6	You are beautiful	23.0
7	Amour	20.0
8	Simply red	26.0
13	Made for mum	20.0
16	Summer sun	22.0

Range search condition

- Also a negated version, NOT BETWEEN
- BETWEEN does not add much to the expressive power of SQL, because

```
SELECT *  
FROM bouquet  
WHERE price >= 20 AND price <= 30;
```



bouquet_id	bouquet_name	price
1	A New Day	23.5
2	The flower of love	20.5
5	Pretty in pink	24.5
6	You are beautiful	23.0
7	Amour	20.0
8	Simply red	26.0
13	Made for mum	20.0
16	Summer sun	22.0

Set membership condition (IN/NOT IN)

- List all bouquet with value 20 and 26

```
SELECT *  
FROM bouquet  
WHERE price IN (20,26);
```



bouquet_id	bouquet_name	price
7	Amour	20.0
8	Simply red	26.0
13	Made for mum	20.0

Set membership search condition

- There is a negated version (NOT IN)
- IN does not add much to the expressive power of SQL, because

SELECT *

FROM bouquet

WHERE price = 20 or price = 26;

- IN is more efficient particularly if the set contains many values

Pattern match

- Searching for the content of strings (Text)
- Not as strict as strict equality (=)
- Many uses
 - Search product catalogues
 - Search keywords
 - Search partial keywords (e.g. all people with first names starting with “Al-”)
 - Alexander
 - Ali
 - Alex
- The LIKE keyword performs string comparison in the WHERE statement
- The LIKE keyword is not case-sensitive

Pattern match search condition

- SQLite has two special pattern matching symbols also known as WILD CARDS:
 - % (percent): any sequence of zero or more characters
 - _ (underscore): any single character
 - Address LIKE 'H%' means the first character must be H, but the rest of the string can be anything
 - Address LIKE 'H____' means that there must be exactly four characters in the string, the first of which must be an H
 - LIKE '%a%' means a sequence of characters of any length containing 'a'

Pattern match search condition (LIKE/NOT LIKE)

- Find all bouquets with the letter 'a' in their name

```
SELECT *  
FROM bouquet  
WHERE bouquet_name LIKE  
'%a%';
```



bouquet_id	bouquet_name	price
1	A New Day	23.5
3	Passion	13.0
6	You are beautiful	23.0
7	Amour	20.0
9	Be my Valentine	13.5
11	Ladies Day	12.5
13	Made for mum	20.0
14	Heaven scent	5.0
15	Pastel perfection	8.0

Like examples (1)



LIKE Operator	Description
WHERE bouquet_name LIKE 'a%'	Finds any values that start with "a"
WHERE bouquet_name LIKE '%a'	Finds any values that end with "a"
WHERE bouquet_name LIKE '%or%'	Finds any values that have "or" in any position
WHERE bouquet_name LIKE '_r%'	Finds any values that have "r" in the second position
WHERE bouquet_name LIKE 'a_%'	Finds any values that start with "a" and are at least 2 characters in length
WHERE bouquet_name LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE bouquet_name LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

Like examples (2)

- Finds bouquet that starts with 'a'

SELECT * FROM bouquet WHERE bouquet_name LIKE 'a%';

bouquet_id	bouquet_name	price
1	A New Day	23.5
7	Amour	20.0

- Finds bouquet that starts with 'a' and are at least 3 characters in length

SELECT * FROM bouquet WHERE bouquet_name LIKE 'a%_ _%';

SELECT * FROM bouquet WHERE bouquet_name LIKE 'a__%';

bouquet_id	bouquet_name	price
1	A New Day	23.5
7	Amour	20.0

NULL search condition (IS NULL/IS NOT NULL)

- List the details of all customers where a phone has not been supplied

SELECT * FROM customer Where phone IS NULL;

Cust_id	name	phone	age	address
7	Nyla Noor	NULL	25	38 Church Road Guildford GU43 6UC

- Negated version (IS NOT NULL) can test for non-null values

SELECT * FROM customer Where phone IS NOT NULL;

Cust_id	name	phone	age	address
1	Ali Rehman	+4475567855344	21	12 hillwood drive Nottingham NG8 6YU
2	Ali Rehman	+4475567855344	45	65 oxford drive Nottingham NG8 6UU
3	Kristian Liam	+4475567845644	31	176 Groove drive London GH4 6TY
4	Lizzi Young	+4474463545024	34	5 Victoria Street Leicester LE67 9YH
5	Ashar Junaid	+4455784186264	65	14 Manor Road Bath BA78 5PA
6	Rebbaca Umar	+4444784553242	23	545 North Street Perth PH80 9XP
8	Ashar Junaid	+4455784186264	56	82 Melbourne Road Nottingham NG8 5TH

Sorting results (ORDER BY Clause)

- In general, the rows of an SQL query result table are not arranged in any particular order
 - Although some DBMSs may use a default ordering based, for example, on a primary key
 - We can use ORDER BY clause to sort query results
 - Sorting in ASC or DESC order on a column(s)
 - Single column ordering or multiple column ordering
 - ORDER BY clause must always be the last clause of the SELECT statement

Sorting results (ORDER BY Clause)

- **SELECT * FROM** customer **ORDER BY** name; (By default ASC order)

Cust_id	name	phone	age	address
1	Ali Rehman	+4475567855344	21	12 hillwood drive Nottingham NG8 6YU
2	Ali Rehman	+4475567855344	45	65 oxford drive Nottingham NG8 6UU
5	Ashar Junaid	+4455784186264	65	14 Manor Road Bath BA78 5PA
8	Ashar Junaid	+4455784186264	56	82 Melbourne Road Nottingham NG8 5TH
3	Kristian Liam	+4475567845644	31	176 Groove drive London GH4 6TY
4	Lizzi Young	+4474463545024	34	5 Victoria Street Leicester LE67 9YH
7	Nyla Noor	<i>NULL</i>	25	38 Church Road Guildford GU43 6UC
6	Rebbaca Umar	+4444784553242	23	545 North Street Perth PH80 9XP

- **SELECT * FROM** customer **ORDER BY** name **DESC**;

Cust_id	name	phone	age	address
6	Rebbaca Umar	+4444784553242	23	545 North Street Perth PH80 9XP
7	Nyla Noor	<i>NULL</i>	25	38 Church Road Guildford GU43 6UC
4	Lizzi Young	+4474463545024	34	5 Victoria Street Leicester LE67 9YH
3	Kristian Liam	+4475567845644	31	176 Groove drive London GH4 6TY
5	Ashar Junaid	+4455784186264	65	14 Manor Road Bath BA78 5PA
8	Ashar Junaid	+4455784186264	56	82 Melbourne Road Nottingham NG8 5TH
1	Ali Rehman	+4475567855344	21	12 hillwood drive Nottingham NG8 6YU
2	Ali Rehman	+4475567855344	45	65 oxford drive Nottingham NG8 6UU

Sorting results (ORDER BY Clause)

SELECT * FROM customer **ORDER BY** name **DESC**, age;

- It orders customers by name in DESC order, but if some rows have the same name, it orders them by age in ASC order

Cust_id	name	phone	age	address
6	Rebbaca Umar	+4444784553242	23	545 North Street Perth PH80 9XP
7	Nyla Noor	<i>NULL</i>	25	38 Church Road Guildford GU43 6UC
4	Lizzi Young	+4474463545024	34	5 Victoria Street Leicester LE67 9YH
3	Kristian Liam	+4475567845644	31	176 Groove drive London GH4 6TY
8	Ashar Junaid	+4455784186264	56	82 Melbourne Road Nottingham NG8 5TH
5	Ashar Junaid	+4455784186264	65	14 Manor Road Bath BA78 5PA
1	Ali Rehman	+4475567855344	21	12 hillwood drive Nottingham NG8 6YU
2	Ali Rehman	+4475567855344	45	65 oxford drive Nottingham NG8 6UU

Limit

- SQLite LIMIT clause is used to limit the data amount returned by the SELECT statement.
- Show top 3 highest priced bouquets
SELECT * FROM bouquet ORDER BY price DESC LIMIT 3;

bouquet_id	bouquet_name	price
4	I love you	33.0
8	Simply red	26.0
5	Pretty in pink	24.5

Aggregate Functions

- The main aggregate functions are:
 - COUNT: returns the number of values in a specified column
 - SUM: returns the sum of the values in a specified column
 - AVG: returns the average of the values in a specified column
 - MIN: returns the smallest value in a specified column
 - MAX: returns the largest value in a specified column

Aggregate Functions

- These functions operate on a single column of a table and return a single value
- Apart from COUNT(*), each function eliminates nulls first and operates only on the remaining non-null values
- COUNT(*) is a special use of COUNT, which counts all the rows of a table, regardless of whether nulls or duplicate values occur
- If we want to eliminate duplicates before the function is applied, we use the keyword DISTINCT before the column name in the function
- It is important to note that an aggregate function can be used only in the
 - SELECT list
 - HAVING clause
- It is incorrect to use it elsewhere

SQL Aggregate Functions Examples

- Use of COUNT(*)
- How many bouquet price is more than 20? Display result as 'Bouquet Count'.

```
SELECT COUNT(*) AS 'Bouquet Count'  
FROM bouquet  
WHERE price > 20;
```

Bouquet Count
7

SQL aggregate functions examples

- Use of COUNT and SUM
- Find the total number of customers as 'Customer Count' and the sum of their ages as 'Ages Sum' whose has a in their name

```
SELECT COUNT(cust_id) AS 'Customer Count',  
SUM(age) AS 'Ages Sum'  
FROM customer  
WHERE name like '%a%';
```

Customer Count	Ages Sum
7	266

SQL Aggregate Functions Examples

- Use of MIN, MAX, AVG
- Find the minimum, maximum, and average price bouquet as 'Minimum Price', 'Maximum Price', 'Average Price'.

```
SELECT MIN(price) AS 'Minimum Price',  
MAX(price) AS 'Maximum Price',  
AVG(price) AS 'Average Price'  
FROM  
bouquet;
```

Minimum Price	Maximum Price	Average Price
5.0	33.0	17.3823529411765

Dates example

- List the greeting messages as 'Special Message' for orders placed in year 2022. Sort the results in ascending order.

```
SELECT greeting_message as 'Special Message'  
from bouquet_order  
WHERE  
orderdate like '%-%-2022'
```

- List the greeting messages as 'Special Message' for orders placed after 12-12-2020. Sort the results in ascending order.

```
SELECT greeting_message as 'Special Message'  
from bouquet_order  
WHERE  
orderdate > '12-12-2020'
```

SQL String Functions :

COMP1004-DBI

String Functions

Function Name	Purpose	Example
	Concat two strings	Select name " phone from customer
Substr substr(string, start, length)	Find substring	Select substr(name,1,4) from customer
Instr(string, substring)	searches a substring in a string and return the position as integer	Select instr(name, 'a') from customer
length(data)	Give length of string	Select length (name) from customer
lower(string)	converted the string to lowercase	Select lower (name) from customer

String Functions

Function Name	Purpose	Example
<code>upper(string)</code>	converted the string to uppercase	Select <code>upper(name)</code> from customer
<code>ltrim(string,[character])</code>	remove all the characters at the beginning of the string.	SELECT <code>ltrim('£599.99', '£')</code> as result;
<code>rtrim(string,[character])</code>	remove all the characters at the end of the string.	SELECT <code>ltrim('599.99 Pound', 'Pound')</code> as result;
<code>REPLACE(string,pattern,replacement)</code>	replace all occurrences of a specified string with another string.	Select <code>replace(phone, '+44', '0')</code> from customer
<code>trim(string, character)</code>	removes specified characters at the beginning and the end of a string.	SELECT <code>trim('£599.9£', '£')</code> as result;

Group By

- SQLite GROUP BY clause is used in collaboration with the SELECT statement to arrange identical data into groups
- It groups the data from the SELECT table(s) and produces a single summary row for each group
- When the WHERE clause is used with GROUP BY, the WHERE clause is applied first, then groups are formed from the remaining rows that satisfy the search condition

Group By example

- List down the bouquet id as 'Bouquet ID' and the number of flowers as 'Flower Count' in each bouquet id. Sort the results in ascending order of bouquet id

```
SELECT bouquet_id as 'Bouquet ID',count(flower_id) as 'Flower Count'  
FROM bouquet_flower  
GROUP BY bouquet_id  
ORDER BY bouquet_id;
```

Having

- HAVING clause is designed for use with GROUP BY to restrict groups that appear in result table
- Similar to WHERE, but WHERE filters individual rows whereas HAVING filters groups
- Column names in HAVING clause must also appear in the GROUP BY list or be contained within an aggregate function
- In practice, the search condition in the HAVING clause always includes at least one aggregate function
- Remember that aggregate functions cannot be used in the WHERE clause

Having example

- List down the bouquet id as 'Bouquet ID' and the number of flowers as 'Flower Count' in each bouquet id. Show only bouquet(s) which have more than three flowers. Sort the results in ascending order of bouquet id

Having example

- List down the bouquet id as 'Bouquet ID' and the number of flowers as 'Flower Count' in each bouquet id. Show only bouquet(s) which have more than three flowers. Sort the results in ascending order of bouquet id.

```
SELECT bouquet_id as 'Bouquet ID',count(flower_id) as 'Flower Count'  
FROM bouquet_flower  
GROUP BY bouquet_id  
HAVING count(flower_id) >3  
ORDER BY bouquet_id;
```

SQL JOIN :

COMP1004-DBI

Cartesian Product

$A = \{x, y, z\}$

$B = \{1, 2, 3\}$

$A \times B = \begin{matrix} (x, 1) & (x, 2) & (x, 3) \\ (y, 1) & (y, 2) & (y, 3) \\ (z, 1) & (z, 2) & (z, 3) \end{matrix}$

`SELECT * FROM A, B;`

A		B		
x	X	1	=	
y		2		
z		3		

Join

- Joins are used when you want to fetch data from more than one tables
- WARNING: having more than one table creates a cross-product! (cross-join)
- You need to use the WHERE statement or particular join to filter the results
- There might be ambiguities in the column-list if more than one table has the same column
 - You can resolve it by referencing columns with the table name:
TableName.ColumnName

```
SELECT  
[DISTINCT | ALL] <column-list>  
FROM <table-names>  
[WHERE <condition>]  
[ORDER BY <column-list>]  
[GROUP BY <column-list>]  
[HAVING <condition>]
```


Example problem

Display a list of the first and last names of all students achieving passing marks along with those marks

```
SELECT
    First, Last, Mark
FROM Student, Grade
WHERE
    (Student.ID =
     Grade.ID) AND
    (Mark >= 40)
```

Student

ID	First	Last
S103	John	Smith
S104	Mary	Jones
S105	Jane	Brown
S106	Mark	Jones
S107	John	Brown

Grade

ID	Code	Mark
S103	DBS	72
S103	IAI	58
S104	PR1	68
S104	IAI	65
S106	PR2	43
S107	PR1	76
S107	PR2	60
S107	IAI	35

Example problem

SELECT ... FROM Student, Grade WHERE...

Are matched
with the first
entry from
the Student
table...

ID	First	Last	ID	Code	Mark
S103	John	Smith	S103	DBS	72
S103	John	Smith	S103	IAI	58
S103	John	Smith	S104	PR1	68
S103	John	Smith	S104	IAI	65
S103	John	Smith	S106	PR2	43
S103	John	Smith	S107	PR1	76
S103	John	Smith	S107	PR2	60
S103	John	Smith	S107	IAI	35
S104	Mary	Jones	S103	DBS	72
S104	Mary	Jones	S103	IAI	58
S104	Mary	Jones	S104	PR1	68
S104	Mary	Jones	S104	IAI	65
S104	Mary	Jones	S		43

All of the
entries from
the Grade
table

And then
with the
second...

and so on

Example problem

```
SELECT ... FROM Student, Grade
WHERE (Student.ID = Grade.ID) AND ...
```

ID	First	Last	ID	Code	Mark
S103	John	Smith	S103	DBS	72
S103	John	Smith	S103	IAI	58
S104	Mary	Jones	S104	PR1	68
S104	Mary	Jones	S104	IAI	65
S106	Mark	Jones	S106	PR2	43
S107	John	Brown	S107	PR1	76
S107	John	Brown	S107	PR2	60
S107	John	Brown	S107	IAI	35

Student.ID Grade.ID

Example problem

```
SELECT ... FROM Student, Grade
WHERE (Student.ID = Grade.ID) AND (Mark >= 40)
```

ID	First	Last	ID	Code	Mark
S103	John	Smith	S103	DBS	72
S103	John	Smith	S103	IAI	58
S104	Mary	Jones	S104	PR1	68
S104	Mary	Jones	S104	IAI	65
S106	Mark	Jones	S106	PR2	43
S107	John	Brown	S107	PR1	76
S107	John	Brown	S107	PR2	60

Example problem

```
SELECT First, Last, Mark FROM Student, Grade
WHERE (Student.ID = Grade.ID) AND (Mark >= 40)
```



First	Last	Mark
John	Smith	72
John	Smith	58
Mary	Jones	68
Mary	Jones	65
Mark	Jones	43
John	Brown	76
John	Brown	60

Join example

- List down the customer name having a in their names as 'Customer Name', Customer order date as 'Order Date' and the total amount of the order as 'Order Amount'. Sort the results according to order amount.

```
SELECT name as 'Customer Name',orderdate as 'Order Date',amount as  
'Order Amount'  
FROM customer,bouquet_order  
WHERE  
customer.Cust_id = bouquet_order.cust_id  
ORDER BY amount;
```

Again, aliases

- Aliases are temporary names for tables or columns
- Created using **AS**

```
SELECT ID AS "Student ID" FROM Student
```

Student ID
S103
S104
S105
S106
S107

- Aliases can be useful
 - If you have lots of table (more readable SQL)
 - Column names are obscure
- Aliases are sometimes used to obfuscate SQL code by hiding the real names of columns and tables
- Also, very useful for self-joins

Table Alias

- Table alias could also be defined as

SELECT * FROM

Table1 a, Table2 b

Where a.columnname = b.coulmunname

Aliases – an example

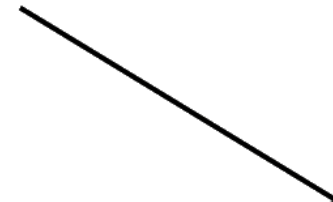
Employee

ID	Name
123	Faisal
124	Sarah

WorksIn

ID	Department
123	Marketing
124	Sales
124	Marketing

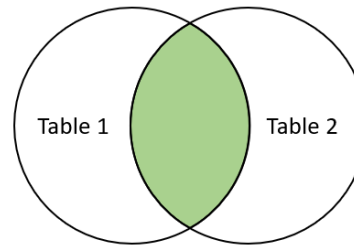
```
SELECT
    E.ID AS "Employee Number",
    E.Name, W.Department
FROM
    EMPLOYEE AS E,
    WorksIn AS W
WHERE
    E.ID = W.ID;
```



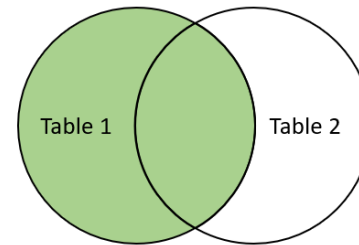
Employee Number	Name	Department
123	Faisal	Marketing
124	Sarah	Sales
124	Sarah	Marketing

SQLite Joins

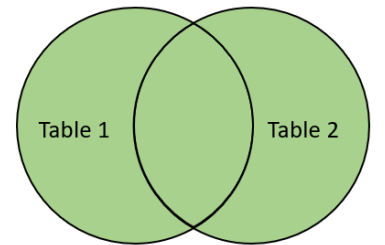
- Cross join
 - Compute the cartesian product of given tables
- Inner Join
 - INNER JOIN clause matches each row from the 1st table with every row from the 2nd table based on the join condition e.g. (table1.id= table2.id)
- Left join
 - The LEFT JOIN clause selects data starting from the left table and matching rows in the right table based on the join condition
 - Preserves information in the leftmost table
 - For not matching rows, padd Null



JOIN



LEFT JOIN



CROSS JOIN

SQLite Joins

- Cross join

```
Select column_list from table1 cross join table2;
```

```
Select column_list from table1,table2;
```

- Inner Join

```
Select column_list from table1 inner join table 2  
ON condition
```

```
If tables have same name:  
Select column_list from table1 inner join table 2  
using (columnname)
```

- Left Join

```
Select column_list from table1 left join table 2  
ON condition
```

```
If tables have same column name:  
Select column_list from table1 left join table 2  
using (columnname)
```

Cross join example

- Compute a cross join of customer and bouquet_order tables.

SELECT *

FROM customer

CROSS JOIN

bouquet_order

Inner join example (with using)

- List down the customer name having a in their names as 'Customer Name', Customer order date as 'Order Date' and the total amount of the order as 'Order Amount'. Sort the results according to order amount

SELECT name as 'Customer Name', orderdate as 'Order Date', amount as 'Order Amount'

FROM customer **INNER JOIN** bouquet_order

USING (cust_id)

WHERE

name like '%a%'

ORDER BY amount

Inner join example (with ON)

- List down the customer name having a in their names as 'Customer Name', Customer order date as 'Order Date' and the total amount of the order as 'Order Amount'. Sort the results according to order amount

SELECT name as 'Customer Name', orderdate as 'Order Date', amount as 'Order Amount'

FROM customer **INNER JOIN** bouquet_order

ON customer.Cust_id = bouquet_order.cust_id

WHERE

name like '%a%'

ORDER BY amount

Left join example

- List down the customer names ,Customer order date as 'Order Date' and the total amount of the order as 'Order Amount'.If customer don't have any order show Null in the order date and order amount. Sort the results according to order amount

SELECT name as 'Customer Name',orderdate as 'Order Date',amount as 'Order Amount'

FROM customer left JOIN bouquet_order

USING (cust_id)

ORDER BY amount

Inner join example (with ON and table alias)

- List down the customer name having a in their names as 'Customer Name', Customer order date as 'Order Date' and the total amount of the order as 'Order Amount'. Sort the results according to order amount

```
SELECT c.name as 'Customer Name',bo.orderdate as 'Order  
Date',bo.amount as 'Order Amount'  
FROM customer c INNER JOIN bouquet_order bo  
ON c.Cust_id = bo.cust_id  
WHERE  
name like '%a%'  
ORDER BY amount
```


Subqueries



Subqueries

- A subquery is a SELECT statement nested in another statement

```
SELECT Column_name  
FROM table1  
WHERE  
columnname =
```



Outer Query

```
( SELECT column FROM table2 );
```



Inner Query

Subquery example

- List bouquet name as 'Bouquet Name',bouquet price as 'Bouquet Price' of the bouquet having Bergamot flowers. Sort the results according to the price.

```
SELECT bouquet_name as 'Bouquet Name' ,price as 'Bouquet Price'  
FROM  
bouquet b,bouquet_flower bf  
WHERE  
bf.flower_id = 2 and b.bouquet_id = bf.bouquet_id  
ORDER BY price;
```

Write queries for following

- Task1: List bouquet name as 'Bouquet Name',bouquet price as 'Bouquet Price' of the bouquet having Bergamot or Blossoming Oregano flowers. Sort the results according to the price.
- Task 2: List the customer name as 'Customer Name', recipient address as 'Address', ordered bouquet name as 'Bouquet Name', greeting message as 'Message' where the age of customer is greater than 20.Sort the results according to the bouquet name in descending order.

COMP1004-DBI

Databases and Interfaces

Module conveners:

Nazia Hameed, Kai Xu